

transformations

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

aggregate_function_over_time

```
aggregate_function_over_time(df, time_field, field_name, aggregate='avg', time_resolution='1 day')
```

Function to process a PySpark DataFrame, computing a given aggregate function on a given field over time windows defined by a time resolution.

Parameters:

Name	Type	Description	Default
df		PySpark DataFrame. It should contain at least two columns, one with the value of time_field as column name and another with the value of field_name.	required
time_field		name of the PySpark DataFrame column associated to the time field.	required
field_name		name of the PySpark DataFrame column associated to the field used to calculate the aggregation over.	required
aggregate		aggregation function to be used. The available functions are "avg", "max", "min", "sum", "count". UDF created with pyspark.sql.functions.pandas_udf() may also be used. See https://spark.apache.org/docs/latest/api/python/pyspark.sql.html#pyspark.sql.GroupedData for more details. (default: avg).	'avg'
time_resolution		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the windowDuration parameter of pyspark.sql.functions.window are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'

Returns:

Type	Description
DataFrame	Pandas DataFrame with two columns. One is the time field column, it has the value of time_field as the column name and contains the start datetime for each time window. The second column contains the values of the aggregation function and its name is the name of the aggregation function that was used.

count_top_n_over_time [↗](#)

```
count_top_n_over_time(df, time_field, field_name, top_n=5, time_resolution='1 day')
```

Function to process a PySpark DataFrame, computing the counts of the top n most frequent categories of a given string field over a time windows defined by a time resolution.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		Pyspark DataFrame. It should contain at least two columns, one with the value of time_field as column name and another with the value of field_name.	<i>required</i>
<code>time_field</code> ↗		name of the PySpark DataFrame column associated to the time field.	<i>required</i>
<code>field_name</code> ↗		name of the PySpark DataFrame column associated to the string field used to calculate the top N values aggregation over time.	<i>required</i>
<code>top_n</code> ↗		the number of the top most frequent categorical values to be drawn.	5
<code>time_resolution</code> ↗		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the windowDuration parameter of pyspark.sql.functions.window are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'

Returns:

Type	Description
Data Frame	Pandas Dataframe with three columns. One is the time field column, it has the value of time_field as the column name and contains the start datetime for each time window. The second contains the frequency counts of the top n values of the string field, over that time window, and is called "count". Lastly, the third contains the name of that categorical value, and the column name is the value of field_name.

datetime_to_unix_epoch_int [↗](#)

```
datetime_to_unix_epoch_int(df, col=None)
```

fix_out_of_bounds_timestamp [↗](#)

```
fix_out_of_bounds_timestamp(time_series)
```

Fixes the time series to be within bounds.

Parameters:

Name	Type	Description	Default
<code>time_series</code> ↗	Series	Pandas.Series representing time.	<i>required</i>

Returns:

Type	Description
Series	series where out of bounds values are fixed to the bound limits.

get_quantiles [↗](#)

```
get_quantiles(df_hist, q=(0.0, 0.25, 0.5, 0.75, 1.0), half_bin_width=0,
is_timestamp_field=False) -> Optional[DataFrame]
```

Computes approximate quantiles from a histogram in its DataFrame representation.

Parameters:

Name	Type	Description	Default
<code>df_hist</code> ↗		histogram as a pandas DataFrame.	<i>required</i>
<code>q</code> ↗		float or tuple/list of floats with quantiles to compute. Values must be between 0 and 1. Default is (0.0, 0.25, 0.5, 0.75, 1.0).	(0.0, 0.25, 0.5, 0.75, 1.0)
<code>half_bin_width</code> ↗		half of the bin width. It will be used to correct the quantile approximation to give the center of the bin. Default: 0 (no correction).	0
<code>is_timestamp_field</code> ↗		boolean indicating if it is a datetime field (default: False).	False

Returns:

Type	Description
DataFrame	DataFrame with a single row containing the quantiles.

get_sampled_df [↗](#)

```
get_sampled_df(data, n_sample=500, balance=True, target=(None, None))
```

Returns a subsample of a PySpark/Pandas dataframe as a Pandas dataframe.

Parameters:

Name	Type	Description	Default
<code>data</code> ↗		Either a PySpark or a Pandas DataFrame.	<i>required</i>
			500

Name	Type	Description	Default
<code>n_sample</code> ↗		number of rows to sample from the DataFrame. If negative, all instances will be sampled. Default: 500.	
<code>balance</code> ↗		boolean indicating if the sampled dataframe should be balanced or not with respect to the target field, so as many positives as non-positives will be sampled. Only relevant if target attribute is set. Default: False.	<code>True</code>
<code>target</code> ↗		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. Default: (None, None).	<code>(None, None)</code>

Returns:

Type	Description
<code>DataFrame</code>	Sampled Pandas DataFrame.

`group_by_sum_with_missing` [↗](#)

```
group_by_sum_with_missing(df, groupby_cols)
```

Applies a group-by sum keeping track of missing values in group-by columns

`histogram2df_fraud_rate` [↗](#)

```
histogram2df_fraud_rate(histogram_df, field_type, target_field, target_value,
cat_threshold=None, time_resolution=None)
```

Transforms the given histogram into a convenient Pandas DataFrame including a fraud rate column.

Parameters:

Name	Type	Description	Default
<code>histogram_df</code> ↗		Pandas DataFrame representation of a histogram.	<i>required</i>
<code>target_field</code> ↗		name of the target field.	<i>required</i>
<code>target_value</code> ↗		value to be considered as 'positives' labels.	<i>required</i>

Name	Type	Description	Default
<code>cat_threshold</code> Altair		number of categories to keep. In case the underlying cardinality is larger, keep the <code>cat_threshold</code> most frequent categories and create a new category 'other' as the aggregation of the rest of categories. The 'missing' category is always treated as a special category, meaning that it will not be merged into 'other' and it will be ignored when finding the <code>cat_threshold</code> most frequent categories. Default is None: keep all categories. Only relevant for string histograms.	None
<code>time_resolution</code> Altair		project time field to given <code>time_resolution</code> . Default is None: keep underlying time resolution. Only relevant for time histograms.	None

Returns:

Type	Description
DataFrame	pandas DataFrame representation of the histogram including a fraud rate column.

histogram2df_numerical [Altair](#)

```
histogram2df_numerical(histogram, breakdowns)
```

Transforms the numerical histogram into a convenient pandas DataFrame for plotting with Altair.

Parameters:

Name	Type	Description	Default
<code>histogram</code> Altair	dict	dictionary representation of a histogram.	<i>required</i>
<code>breakdowns</code> Altair	list	list of breakdown names.	<i>required</i>

Returns:

Type	Description
DataFrame	pandas DataFrame representation of the histogram.

histogram2df_string [Altair](#)

```
histogram2df_string(histogram, breakdowns)
```

Transforms the string histogram into a convenient pandas DataFrame for plotting with Altair.

Parameters:

Name	Type	Description	Default
<code>histogram</code> Altair		dictionary representation of a histogram.	<i>required</i>
<code>breakdowns</code> Altair		list of breakdown names.	<i>required</i>

Returns:

Type	Description
DataFrame	pandas DataFrame representation of the histogram.

histogram2df_time [¶](#)

```
histogram2df_time(histogram, breakdowns)
```

Transforms the time histogram into a convenient pandas DataFrame for plotting with Altair.

Parameters:

Name	Type	Description	Default
histogram ¶	dict	dictionary representation of a histogram.	<i>required</i>
breakdowns ¶	list	list of breakdown names.	<i>required</i>

Returns:

Type	Description
DataFrame	pandas DataFrame representation of the histogram.

normalize_histogram [¶](#)

```
normalize_histogram(df, breakdown\_by=None)
```

Function to normalize the histogram to unity.

Parameters:

Name	Type	Description	Default
df ¶	DataFrame	pandas DataFrame representation of a histogram.	<i>required</i>
breakdown_by ¶	str	string that specifies the breakdown field. The histogram of every breakdown value will be normalized to unity. Default is None: normalize without breakdown.	None

Returns:

Type	Description
DataFrame	Normalized DataFrame representation of the histogram.

quantiles_over_time [¶](#)

```
quantiles_over_time(df, time\_field, field\_name, time\_resolution='1 day')
```

Function to process a PySpark DataFrame, computing the min, max and quantiles of a given field over time windows defined by a time resolution. The quantiles are approximated, yielding a relative error of 0.01%. See https://spark.apache.org/docs/2.3.0/api/sql/index.html#percentile_approx for more details.

Parameters:

Name	Type	Description	Default
<code>df</code> Altair		PySpark DataFrame. It should contain at least two columns, one with the value of <code>time_field</code> as column name and another with the value of <code>field_name</code> .	<i>required</i>
<code>time_field</code> Altair		name of the PySpark DataFrame column associated to the time field.	<i>required</i>
<code>field_name</code> Altair		name of the PySpark DataFrame column associated to the field used to calculate the quantiles over.	<i>required</i>
<code>time_resolution</code> Altair		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the <code>windowDuration</code> parameter of <code>pyspark.sql.functions.window</code> are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'

Returns:

Type	Description
<code>DataFrame</code>	Pandas DataFrame with six columns. One is the time field column, it has the value of <code>time_field</code> as the column name and contains the start datetime for each time window. The others contain the values of the min, max and the 25%, 50% and 75% quantiles. They are called min, 25%, median, 75% and max and appear in this order.

reduce_categorical_histogram [Altair](#)

```
reduce_categorical_histogram(histogram: DataFrame, breakdown_by: str | None = None,
cat_threshold: int | None = None, **kwargs)
```

reduce_histogram [Altair](#)

```
reduce_histogram(histogram: DataFrame, result_type: str, **kwargs) -> DataFrame
```

Function to reduce a histogram DataFrame to the minimal information to pass to Altair.

Parameters:

Name	Type	Description	Default
<code>histogram</code> Altair	<code>DataFrame</code>	DataFrame representation of a histogram.	<i>required</i>
<code>result_type</code> Altair	<code>str</code>	Type of histogram result.	<i>required</i>

Other Parameters:

Name	Type	Description
<code>breakdown_by</code>	<code>str</code>	field to breakdown by. If None (default) then don't breakdown. If the original histogram has multiple breakdown fields, project all fields except the breakdown field of interest.
<code>cat_threshold</code>	<code>int</code>	number of categories to keep. In case the underlying cardinality is larger, keep the <code>cat_threshold</code> most frequent categories and create a new category 'other' as the aggregation of the rest of categories. The 'missing' category is always treated as a special category, meaning that it will not be merged into 'other' and it will be ignored when finding the <code>cat_threshold</code> most frequent categories. Default is None: keep all categories. Only relevant for string histograms.
<code>time_resolution</code>	<code>str</code>	project time field to given <code>time_resolution</code> . Default is None: keep underlying time resolution. Only relevant for time histograms.

Returns:

Type	Description
<code>DataFrame</code>	Reduced histogram.

reduce_numerical_histogram [¶](#)

```
reduce_numerical_histogram(histogram: DataFrame, breakdown_by: str | None = None, **kwargs) -> DataFrame
```

reduce_temporal_histogram [¶](#)

```
reduce_temporal_histogram(histogram: DataFrame, breakdown_by: str | None = None, time_resolution: str | None = None, **kwargs) -> DataFrame
```

temporal_aggregation [¶](#)

```
temporal_aggregation(df, time_field, aggregate, time_resolution='1D', is_timestamp_field=False)
```

Aggregates a dataframe with respect to time.

Parameters:

Name	Type	Description	Default
<code>df</code> ¶		dataframe.	<i>required</i>
<code>time_field</code> ¶		name of the field associated with the temporal dimension.	<i>required</i>
<code>aggregate</code> ¶		aggregation function to be used. Can be any of <code>plots.SUPPORTED_AGGREGATIONS</code> .	<i>required</i>
<code>time_resolution</code> ¶		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.Grouper.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> <code>_</code> . Default: '1D'.	'1D'

Name	Type	Description	Default
<code>is_timestamp_field</code> ↗		boolean indicating if it is a datetime field (default: False).	False

Returns:

Type	Description
DataFrame	Pandas DataFrame with two columns. One is the time field column, it has the value of time_field as the column name and contains the start datetime for each time window. The second column contains the values of the aggregation function and its name is the name of the aggregation function that was used.

temporal_aggregation_all [↗](#)

```
temporal_aggregation_all(df, time_field, time_resolution='1D', is_timestamp_field=False)
```

temporal_quantiles [↗](#)

```
temporal_quantiles(df_hist, time_field, is_timestamp_field=False, time_resolution='1W', half_bin_width=0)
```

Estimates the quantiles of a field with respect to time.

Parameters:

Name	Type	Description	Default
<code>df_hist</code> ↗		dataframe.	<i>required</i>
<code>time_field</code> ↗		name of the field associated with the temporal dimension.	<i>required</i>
<code>is_timestamp_field</code> ↗		boolean indicating if it is a datetime field (default: False).	False
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas . Grouper freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Default: '1W'.	'1W'
<code>half_bin_width</code> ↗		half of the bin width. It will be used to correct the quantile approximation to give the center of the bin. Default: 0 (no correction).	0

Returns:

Type	Description
DataFrame	Pandas DataFrame with six columns. One is the time field column, it has the value of time_field as the column name and contains the start datetime for each time window. The others contain the values of the min, max and the 25%, 50% and 75% quantiles. They are called min, 25%, median, 75% and max and appear in this order.

unpack_breakdowns [¶](#)

```
unpack_breakdowns(df, breakdowns)
```

Unpacks the DataFrame column 'breakdowns' into its components.

Parameters:

Name	Type	Description	Default
df ¶	DataFrame	a pandas DataFrame with a column named 'breakdowns' containing a tuple with the breakdown values.	<i>required</i>
breakdowns ¶	list	list of breakdown names.	<i>required</i>

Returns:

Type	Description
DataFrame	unpacked pandas DataFrame.